

EVENTFLOW

EF-DOC-000

EventFlow Constitution

Version 0.2 | Draft for Product Owner Review

Every Guest Matters. Every Event Flows.

Document Control

Document ID	EF-DOC-000
Title	EventFlow Constitution
Version	0.2
Status	Draft for Product Owner Review
Product	EventFlow
Document Owner	Product Owner
Technical Owner	Solution Architect
Created	2026-08-06
Last Updated	2026-08-06
Review Frequency	At major product-governance changes
Classification	Internal Product Governance
Related Documents	EF-DOC-001, EF-DOC-002, EF-DOC-003

Revision History

Version	Date	Status	Summary
0.1	2026-08-06	Draft	Initial constitutional principles established.
0.2	2026-08-06	Draft for Review	Formalized governance, decision gates, terminology, release obligations, scalability, security, and amendment rules.

Approval Record

Role	Approval	Date
Product Owner	Pending	-
Solution Architect	Prepared	2026-08-06

Preamble

EventFlow is a professional Event Operations Platform intended to manage the complete operational lifecycle of events, from event configuration and invitations through attendee management, communications, seating, reception, check-in, reporting, and future extensions.

This Constitution defines the highest-level, non-negotiable product and engineering principles governing EventFlow. Product requirements, architecture, database design, interfaces, implementation, testing, operations, and documentation must conform to this Constitution unless a formal amendment is approved and recorded.

The Constitution is intentionally stable. Detailed implementation decisions belong in Architecture Decision Records (ADRs), the Product Requirements Document (PRD), architecture specifications, database specifications, and other controlled documents.

Article I - Product Identity and Purpose

1. EventFlow shall be developed as a reusable Event Operations Platform, not as a single-event website, RSVP form, invitation plugin, or event-specific codebase.
2. The platform shall support multiple event categories through configuration and extension rather than event-specific source-code changes.
3. The initial Lui @ 60 implementation is the first production use case and proving ground for EventFlow, but it shall not define or constrain the platform's long-term architecture.
4. EventFlow's product motto is: Every Guest Matters. Every Event Flows.

Article II - Event-Centric Domain Model

1. The Event is the root business entity.
2. Invitations, attendees, communications, venue configurations, tables, seating assignments, check-ins, reports, audit records, and event-scoped settings shall be associated with an Event.
3. Event ownership shall be explicit in the data model to support future multi-event operation without fundamental redesign.
4. Cross-event data access shall be controlled and intentional.

Article III - Invitations and Attendees

1. An Invitation represents the organizer's invitation relationship with a primary guest or invitee group.
2. An Invitation may authorize one or more attendee places.
3. The primary guest and confirmed companions shall be represented as Attendee records when individual identity becomes operationally relevant.
4. Companion names shall not remain permanently embedded only as unstructured text or JSON when seating, check-in, badge generation, reporting, or other attendee-level operations require individual records.
5. Invitation-level and attendee-level responsibilities shall remain distinct.

Article IV - Configuration Over Hard-Coding

1. Event-specific information shall be data or configuration, not source code.
2. Configurable information includes, at minimum, event name, venue, dates, schedules, branding, invitation artwork, theme, organizer contact details, guest-facing wording, deadlines, communication templates, and integration settings.
3. Features that require code edits to change ordinary event information shall be considered architectural defects unless explicitly justified by an ADR.
4. Reusing EventFlow for a new event should primarily require configuration and data import, not software modification.

Article V - Single Source of Truth and Data Ownership

1. Every business fact shall have a defined authoritative owner.
2. Derived data, cached data, dashboards, exports, and reports may reproduce information for presentation or performance but shall not become conflicting sources of truth.
3. Data ownership shall be documented in the Database Design Specification.
4. Database duplication shall be justified by performance, audit, historical, or integration requirements and documented where material.
5. Data migrations shall preserve lineage and existing event data whenever practical.

Article VI - Modular Architecture and Defined Boundaries

1. EventFlow shall be modular.
2. Modules shall have clearly defined responsibilities, interfaces, and dependencies.
3. Communications shall not own seating rules; seating shall not own invitation delivery; reception shall not own event configuration; reporting shall generally read operational data rather than silently mutate it.
4. Shared capabilities shall be implemented as shared services rather than duplicated module logic.
5. Module boundaries shall be documented in the Software Architecture Document.

Article VII - Security, Privacy, and Least Privilege

1. Security and privacy are product requirements, not optional enhancements.
2. EventFlow shall apply least-privilege access controls.

3. Administrative actions shall require appropriate authorization and, where applicable, anti-CSRF protections.
4. External input shall be validated and normalized; rendered output shall be appropriately escaped.
5. Credentials and integration secrets shall not be exposed in source repositories, public pages, logs, or client-side code.
6. Public invitation URLs shall minimize personally identifiable information and use high-entropy, non-guessable tokens or an equivalent secure mechanism.
7. Sensitive operations shall be auditable.
8. Data retention and deletion capabilities shall evolve with product maturity and applicable privacy obligations.

Article VIII - Mobile-First and Accessible User Experience

1. Guest-facing experiences shall be designed mobile-first.
2. Administrative interfaces shall be responsive and optimized for operational efficiency.
3. Public interfaces shall target WCAG-aligned accessibility practices appropriate to the supported platform, including keyboard usability, semantic structure, sufficient contrast, form labels, readable typography, and assistive-technology compatibility.
4. Critical workflows shall minimize unnecessary steps and avoid requiring technical knowledge from guests.
5. Guest-facing errors shall be understandable and recoverable without exposing sensitive internal details.

Article IX - Scalability, Capacity, and Performance

1. Architecture shall be designed to scale from small private events to events with at least several thousand attendees without fundamental redesign of core entity relationships or module boundaries.
2. Capacity assumptions and measurable service targets shall be defined in the PRD and Architecture documents rather than left implicit.
3. Bulk operations shall be designed for batching, idempotency, retryability, and failure isolation where applicable.
4. Database access shall favor indexed, bounded, and paginated queries for growing datasets.

5. Expensive operations such as bulk communications, report generation, imports, and exports should support asynchronous or queued execution when scale justifies it.
6. Performance optimizations shall not compromise correctness, security, or auditability without an explicit accepted trade-off.

Article X - Reliability and Operational Resilience

1. Event-day operations shall prioritize reliability over novelty.
2. Critical workflows such as guest lookup and check-in should degrade gracefully when external integrations are unavailable.
3. Integrations with email, SMS, payment, or other external services shall not become single points of failure for unrelated core operations.
4. Failed external operations shall expose actionable status and support retry where appropriate.
5. Backups, migration safety, and rollback mechanisms shall be part of the release and deployment strategy.

Article XI - Auditability and Observability

1. Material actions should be attributable to an actor, timestamp, event, and relevant business object.
2. Audit-worthy actions include, at minimum, invitation sending, invitation-status changes, attendee changes, table assignments, check-ins, role/permission changes, integration configuration changes, and significant event-setting changes.
3. Operational logs and audit logs shall serve distinct purposes.
4. Logging shall avoid unnecessary storage of secrets or sensitive personal data.
5. Future observability capabilities may include structured logs, health checks, metrics, and error reporting as product maturity requires.

Article XII - Documentation as a Product Deliverable

1. Documentation is part of EventFlow.
2. A release is not complete until applicable documentation is current.
3. Controlled documents include the Constitution, Vision & Design Principles, ADR Register, PRD, Architecture, Database Design, API/Integration Specification, UI/UX Guide, Security Guide, Developer Guide, Testing Strategy, Release Notes, Changelog, migration guidance, and the EventFlow Bible.

4. ADRs shall preserve historical decisions. Accepted ADRs shall not be silently rewritten to conceal subsequent architectural change.
5. A superseded decision shall be documented by a new ADR that references the superseded ADR.
6. Versioned documentation packages shall be archived at defined release milestones.

Article XIII - Requirements and Decision Gates

Before implementation of a material feature, the team shall determine:

1. Whether the feature is covered by an approved or draft PRD requirement.
2. Whether it introduces or changes a significant architectural decision requiring an ADR.
3. Whether it changes the database schema, integration contracts, security posture, capacity assumptions, or user experience standards.
4. Which controlled documents require revision.
5. What acceptance criteria and tests demonstrate completion.

Material work should not bypass these gates merely for short-term convenience.

Article XIV - Versioning, Releases, and Backward Compatibility

1. EventFlow shall use semantic versioning unless superseded by an ADR.
2. Releases shall distinguish major, minor, and patch changes.
3. Breaking changes shall be intentional, documented, migration-aware, and appropriately versioned.
4. Existing event data shall be preserved through upgrades whenever practical.
5. Database schema changes shall use controlled migrations rather than ad hoc manual changes.
6. Each release shall include release notes, a changelog update, applicable migration guidance, and an identified rollback or recovery approach.

Article XV - Engineering Quality

1. Code shall favor clarity, maintainability, testability, and predictable behavior over cleverness.
2. New development shall avoid unnecessary dependencies.
3. Shared logic shall be reused rather than copied.
4. Security-sensitive and business-critical code shall receive proportionate testing.

5. Technical debt may be accepted consciously but shall be documented when it materially affects the roadmap, reliability, security, or maintainability.
6. The codebase should be left in a better state after each release.

Article XVI - Product Governance and Roles

1. The Product Owner owns business priorities, product direction, acceptance, and final product decisions.
2. The Solution Architect owns architectural integrity, technical trade-off analysis, technical documentation, and alignment between product requirements and implementation.
3. Contributors shall follow documented development, review, testing, and release practices.
4. Conflicts between short-term feature delivery and constitutional principles shall be surfaced explicitly rather than resolved silently.
5. Product decisions and architecture decisions shall be recorded in the appropriate controlled documents.

Article XVII - Amendment and Exception Process

1. This Constitution may be amended only when EventFlow's governing principles genuinely need to change.
2. An amendment requires:
 - a documented rationale;
 - impact analysis;
 - Product Owner approval;
 - Solution Architect review;
 - an ADR when the amendment reflects a material architectural decision;
 - an incremented Constitution version.
3. Temporary exceptions shall be documented with scope, rationale, risk, owner, and planned resolution.
4. Convenience alone is insufficient justification for permanent constitutional exceptions.

Article XVIII - Product Motto and Guiding Standard

Every Guest Matters. Every Event Flows.

The motto expresses EventFlow's commitment to both human experience and operational excellence. Product decisions should improve the experience of guests while helping organizers execute events predictably, securely, and efficiently.

Constitutional Interpretation

Where a lower-level document conflicts with this Constitution, the Constitution prevails until the conflict is resolved through an approved amendment or correction.

Where the Constitution is silent, the PRD, accepted ADRs, Architecture Document, and other controlled specifications govern according to their scope.

Next Review

This Constitution shall be reviewed before the first approved EventFlow 1.0 product release and whenever a proposed change materially alters the product's identity, governance, security model, data ownership, or architectural principles.